SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1409 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Jagarajupalli Bhanuprakash Reddy/192524171 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

J.Bhanuprakash Reddy

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
   int 1num = 100;
   float rate@ = 25.5;
   char ch = 'abc';
```

```
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;

    printf("Welcome to Compiler Design);

    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

    float value = 5..6;
    int data = 10@5;

    char c = 'xy';
    int 3value = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.
ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.
ii) Specify which rule of token formation is violated:
iii) Identifier rules
iv) Constant formation
v) String/character literal rules
vi) Operator/symbol rules

(c) Token Correction

i) Rewrite each invalid token into a valid token.
ii) Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.
ii) Discuss whether the compiler:
iii) Stops immediately
iv) Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;
   int total;
   total = x + y * result + 0;
   while(x < 20) {
      printf("%d\n", x);
   }    return 0;}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.

II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

I. Lexical (invalid symbols like @num)

II. Syntax (missing semicolon, brackets)

III. Semantic (function argument mismatch)

IV. Intermediate Code Issues (wrong expression handling/precedence)

V. Optimization Issues (redundant + 0)

VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?

II. Which errors appear only during **execution**?

III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

I. Rewrite the **fully corrected program**.

II. Ensure:

A. Proper syntax

B. Correct semantics

C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.

II. Analyze how **array bounds violation** impacts memory.

III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:

A. Detect runtime errors earlier

B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

Answers :

Q1. ERROR ANALYSIS

The given C program is designed to perform **simple arithmetic operations, function calls, array access, and looping constructs**. However, the program contains multiple errors introduced at different stages of compilation and execution, including **syntax, semantic, logical, and runtime errors**. The purpose of this analysis is to identify each error, determine its nature and compiler phase, analyze whether it is detected during compilation or execution, correct the program, and evaluate the impact of these errors on program behavior.

(a) Error Identification

Line	Error	Nature of Error	Explanation
51	i <= n	Logical / Runtime	For an array of 5 elements, valid indexes are 0–4. When i = 5, arr[5] is accessed.
52	arr[i] when i = 5	Runtime	Accessing arr[5] is outside the allocated array boundary.
58	Missing ;	Syntax	The array declaration must end with a semicolon.
61	sumArray(arr)	Semantic	sumArray() requires two arguments: arr and n, but only one is supplied.
64	Missing ; after printf()	Syntax	The printf() statement must end with a semicolon.
68	a / b, where b = 0	Runtime	Division by zero is invalid during execution.
70–71	Uninitialized pointer p	Runtime	p does not point to a valid memory location before dereferencing.
71	*p = 20	Runtime	Dereferencing an uninitialized pointer can cause undefined behavior or a crash.

Root Cause Analysis

1. Array-bound error:

The loop uses i <= n. If n = 5, values of i are 0, 1, 2, 3, 4 and 5. But a five-element array has indexes only from 0 to 4. Therefore, the condition should be i < n.

2. Missing semicolon:

C requires a semicolon to terminate declarations and expressions. Therefore, lines 58 and 64 cause syntax errors.

3. Incorrect function call:

The function is declared as:

int sumArray(int arr[], int n)

It requires two parameters. Calling it as:

sumArray(arr);

does not provide the required n argument.

4. Division by zero:

b is explicitly initialized to zero:

int a = 10, b = 0;

Therefore:

a / b

attempts to divide by zero.

5. Invalid pointer usage:

p is declared but not initialized:

int *p;

The statement:

*p = 20;

attempts to write to an unknown memory location.

(b) Classification of Errors

Error	Classification	Compiler Phase
Missing ; at line 58	Syntax	Syntax Analysis
Missing ; at line 64	Syntax	Syntax Analysis
Missing function argument	Semantic	Semantic Analysis
i <= n	Logical	Program logic
arr[5]	Runtime / logical	Execution
Division by zero	Runtime	Execution
Uninitialized pointer	Runtime	Execution
Invalid pointer dereference	Runtime	Execution

Lexical Errors

There are **no major lexical errors**.

(c) Compilation Behavior

Errors detected during compilation

The following can be detected during compilation:

1. Missing semicolon at line 58.
2. Missing semicolon after printf() at line 64.
3. Incorrect number of arguments in sumArray(arr).

The compiler normally reports syntax/semantic diagnostics before producing a valid executable.

Errors that appear during execution

The following may only become apparent when the relevant statements execute:

- Division by zero.
- Invalid array access.
- Dereferencing an invalid/uninitialized pointer.

Why early errors prevent later errors from being detected

Compilation occurs in stages:

Source Code → Lexical Analysis → Syntax Analysis → Semantic Analysis → Code Generation → Execution

If a syntax error occurs, the compiler may not be able to construct the correct syntax tree. Therefore, later stages may not receive enough valid information to analyze the rest of the program.

For example, if the missing semicolon produces a syntax error, the compiler may report that error first and may not perform complete semantic analysis of later statements.

(d) Corrected Program

```
#include <stdio.h>
int sumArray(int arr[], int n) {
   int i, sum = 0;
   for (i = 0; i < n; i++) {
      sum = sum + arr[i];
   }
   return sum;
```

```
}
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int total;
    total = sumArray(arr, 5);
    if (total == 15) {
        printf("Correct Sum\n");
    }
    int a = 10, b = 2;
    int c = a / b;
    printf("Result = %d\n", c);
    int p;
    p = 20;
    printf("Pointer value = %d\n", p);
    return 0;
}
```

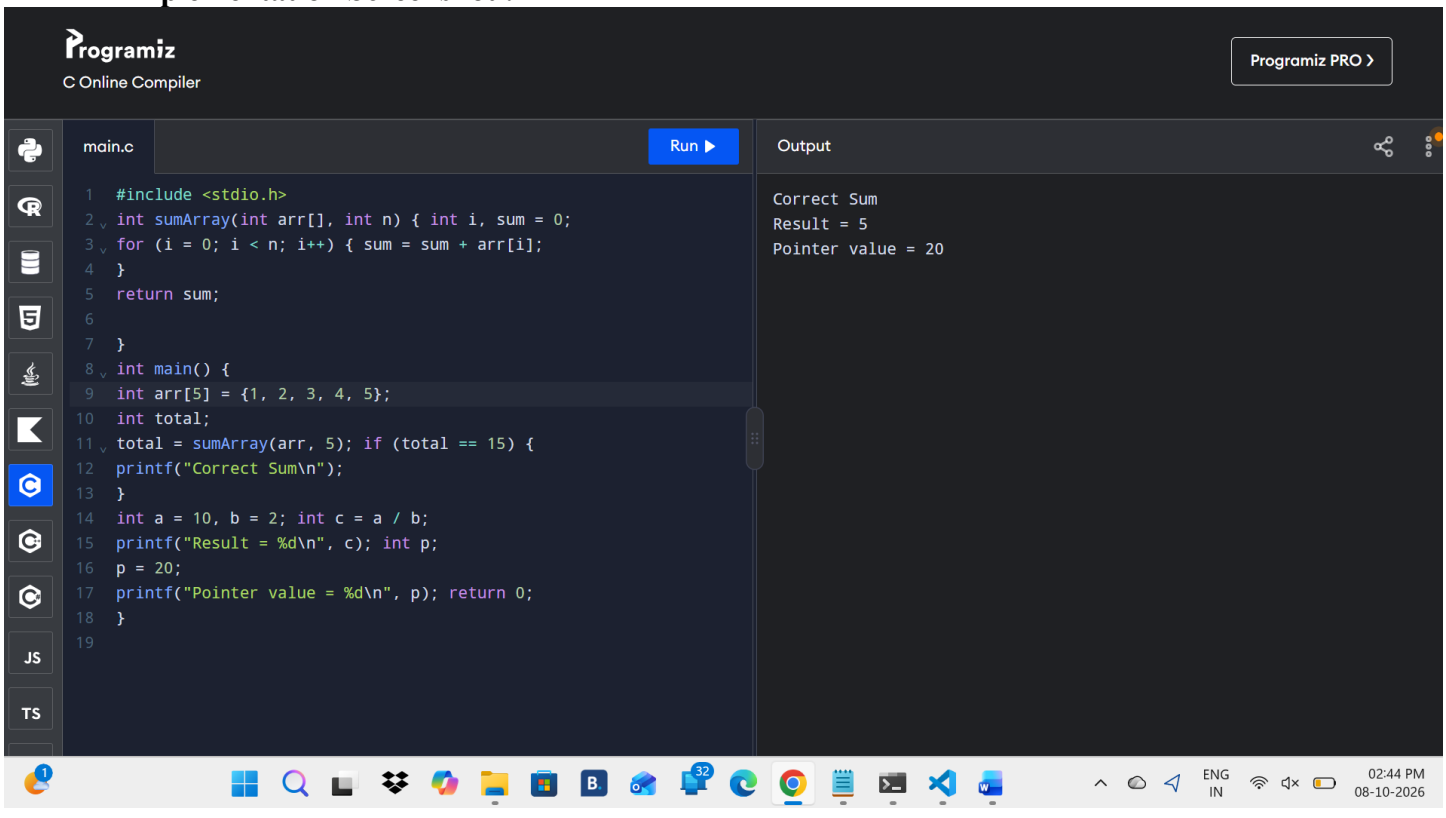
Expected Output

Correct Sum

Result = 5

Pointer value = 20

Implementation Screenshot :



(e) Impact Analysis

1. Division by zero

The statement:

int c = a / b;

with b = 0 attempts an illegal arithmetic operation.

Impact:

- Program may terminate abnormally.
- The result is undefined.
- Depending on the environment, an arithmetic exception may occur.

2. Improper pointer usage

The statement:

int *p;

*p = 20;

uses a pointer without assigning it a valid address.

Impact:

- It can overwrite an unknown memory location.
- It may cause segmentation faults.
- It can corrupt program data.
- It results in undefined behavior.

3. Array bounds violation

The loop:

for(i = 0; i <= n; i++)

allows i to become 5 when n = 5.

Therefore:

arr[5]

is accessed.

Impact:

- Memory outside the array may be accessed.
- It can read unrelated memory.
- It can cause undefined behavior.
- It may result in incorrect output or program crashes.

4. Incorrect function call

sumArray(arr);

does not provide the required n argument.

Impact:

The compiler cannot correctly match the function call with its declaration, resulting in a compilation diagnostic.

(f) Advanced Compiler Perspective

1. Better error messages

A compiler should provide:

- Exact line number.
- Exact location/column.
- Description of the problem.
- Possible reason.
- Suggested correction.

For example:

Error: line 58

Expected ';' after array declaration.

Suggestion: Add ';' at the end of the declaration.

Instead of only:

Syntax error

2. Early detection of runtime errors

Static analysis can identify many possible runtime problems before execution.

For example:

int b = 0;

int c = a / b;

A compiler or static analyzer can warn:

Warning: Division by zero detected.

Similarly, tools can detect:

- Possible null/uninitialized pointer dereference.
- Array-bound violations.
- Uninitialized variables.

3. Improvement 1 – Static Analysis

A static-analysis phase can examine the program without executing it and identify suspicious constructs such as:

arr[5]

when the array contains only five elements.

4. Improvement 2 – Debugging and Sanitizer Support

A compiler can integrate runtime checking tools such as:

- Address checking.
- Undefined-behavior detection.
- Memory-access checking.

These can identify invalid memory accesses and provide detailed diagnostic information.

Q2. LEXICAL ERROR ANALYSIS

The given C program contains multiple **lexical errors** that are intentionally introduced in identifiers, constants, string and character literals, and special symbols. These errors occur during the **lexical analysis phase**, where the compiler converts the source program into a sequence of valid tokens. The purpose of this analysis is to identify the invalid tokens, understand which lexical rules are violated, correct them, and examine how a lexical analyzer detects and reports such errors.

(a) Error Identification

Line	Invalid Token / Construct	Error
130	1num	Invalid identifier
131	rate@	@ is not valid in an identifier
132	'abc'	Invalid character literal for a single character
133	50$20	$ is not a valid C operator/token in this expression
134	num#1	# is not valid inside an identifier
135	%count	% cannot begin an identifier
137	"Welcome to Compiler Design)	Unterminated string literal
139	09	Invalid octal integer constant
140	0xZ12	Invalid hexadecimal constant
141	12.3.4	Malformed floating-point constant
143	"Hello;	Unterminated string literal
147	invalid-id	- is interpreted as an operator, not part of an identifier
149	5..6	Malformed floating-point constant
150	10@5	Invalid @ symbol
152	'xy'	Invalid character literal
153	3value	Invalid identifier

Important note

_valid on line 146 is **valid** because an identifier can begin with an underscore.

(b) Error Classification

1. Identifier Rules

A normal C identifier can contain:

- Letters
- Digits after the first character
- Underscore _

It cannot begin with a digit.

Examples

Invalid:

1num

3value

Correct:

num1

value3

2. Constant Formation

Line 139

int x = 09;

The leading 0 traditionally indicates an octal integer constant in C, but octal digits range only from 0 to 7. Therefore 9 is invalid in that form.

Correct:

int x = 9;

Line 140

int y = 0xZ12;

A hexadecimal constant can contain:

0–9, A–F, a–f

Z is not a hexadecimal digit.

Correct example:

int y = 0xA12;

Line 141

int z = 12.3.4;

A floating-point constant cannot contain two decimal points.

Correct:

int z = 12.34;

3. String and Character Literal Rules

A character literal normally represents a character using single quotes:

'A'

A string uses double quotes:

"Hello"

Therefore:

char c = 'xy';

is not a proper single-character literal.

Correct:

char c = 'x';

Similarly:

char ch = 'abc';

should be:

char ch = 'a';

if only one character is intended.

Unterminated string

printf("Welcome to Compiler Design);

The closing " is missing.

Correct:

printf("Welcome to Compiler Design");

4. Operator/Symbol Rules

Characters such as @, $ and # cannot arbitrarily be inserted into identifiers or expressions.

Examples:

rate@

num#1

10@5

contain invalid uses of these symbols.

(c) Token Correction

Invalid	Correct Token
1num	num1
rate@	rate
'abc'	'a'
50$20	50 + 20
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xZ12	0xA12
12.3.4	12.34
"Hello;	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10 + 5
'xy'	'x'
3value	value3

The corrections preserve the intended meaning where possible while converting invalid lexical constructs into valid C tokens.

(d) Compiler Behavior Analysis

How the lexical analyzer processes errors

The lexical analyzer, or scanner, reads the source program character by character and groups characters into tokens.

For example:

int value = 10;

is recognized approximately as:

int → KEYWORD

value → IDENTIFIER

= → OPERATOR

10 → CONSTANT

; → DELIMITER

When the scanner encounters something that does not form a valid token, it reports a lexical error.

For example:

3value

starts with a digit, so the scanner initially recognizes a numeric token. When it encounters v, the sequence cannot form a valid ordinary numeric constant, leading to an invalid tokenization/diagnostic.

Does the compiler stop immediately?

Not necessarily.

A compiler can use **error recovery** to continue scanning and report multiple errors in one compilation.

For example:

Error at line 130: Invalid identifier '1num'

Error at line 131: Invalid character '@'

Error at line 140: Invalid hexadecimal constant

This is better than stopping after the first error because the programmer can fix several problems at once.

However, severe lexical errors, particularly an unterminated string literal, can cause the scanner to lose track of token boundaries and produce additional cascading errors.

Corrected Code :

```
#include <stdio.h>
int main() {
   int num1 = 100;
   float rate = 25.5;
   char ch = 'a';
   int total = 50 + 20;
   int num1_value = 30;
   int count = 40;
   printf("Welcome to Compiler Design");
   int x = 9;
   int y = 0xA12;
   int z = 12.34;
```

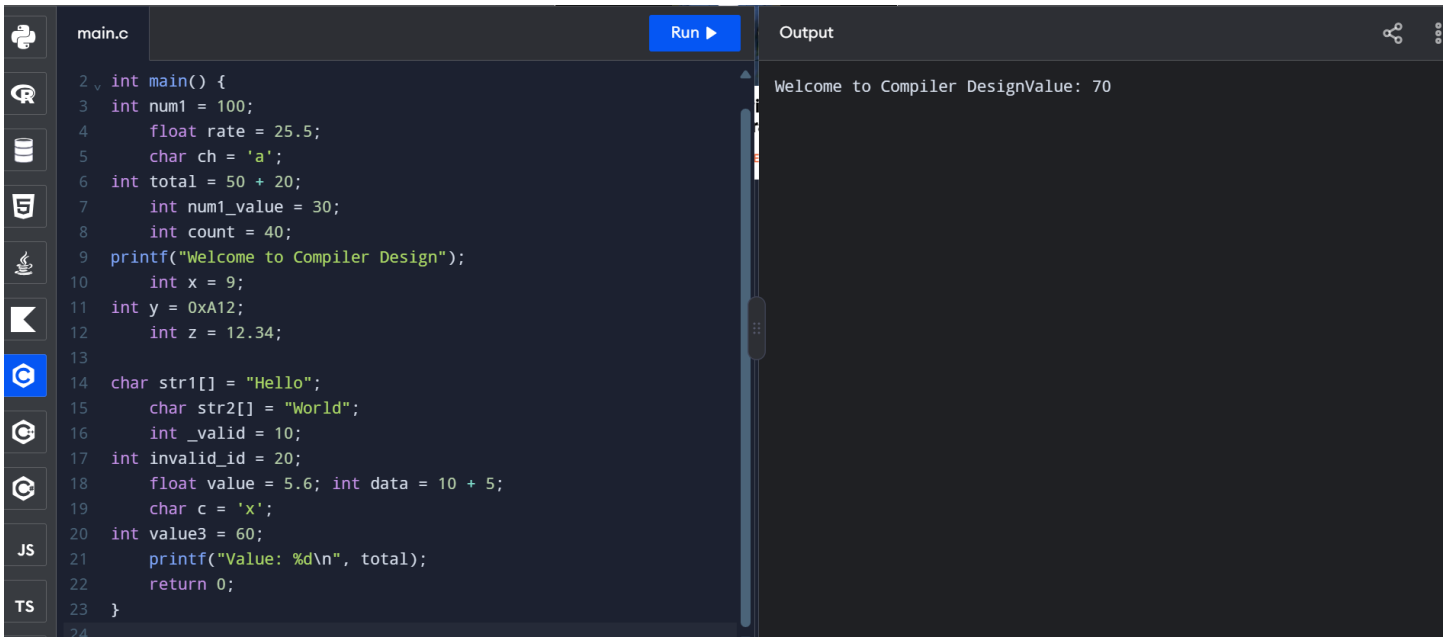
```
    char str1[] = "Hello";
    char str2[] = "World";
    int _valid = 10;
    int invalid_id = 20;
    float value = 5.6;
    int data = 10 + 5;
    char c = 'x';
    int value3 = 60;
    printf("Value: %d\n", total);
    return 0;
}
```

Expected Output

Welcome to Compiler Design

Value: 70

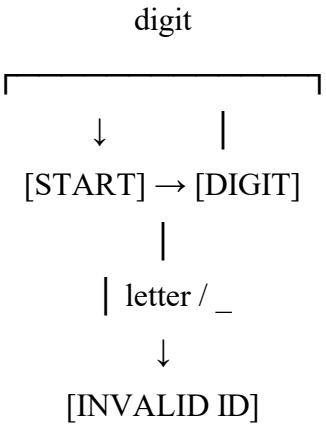
Implementation Screenshot :



(e) Advanced Thinking

1. DFA idea for invalid identifiers beginning with digits

A simple DFA can be designed as follows:



Rules:

- Start state accepts a letter or _ for a valid identifier.
- If the first character is a digit, enter a numeric state.
- If a letter or underscore follows the initial digit in an identifier-like sequence, report an invalid identifier.

Valid

value

_value

value1

Invalid

1value

3num

2. Better beginner-friendly error messages

Instead of:

Invalid token

the compiler should provide:

Line 153: Invalid identifier '3value'.

Identifiers cannot start with a digit.

Try: value3

This provides:

Location + Problem + Reason + Suggested correction

which is much more useful to beginners.

Q3. MULTI-PHASE COMPILER ERROR ANALYSIS

The given C program contains errors distributed across different phases of compilation, including **lexical, syntax, semantic, intermediate-code, optimization, and runtime phases**. The program includes invalid symbols, missing semicolons, incorrect function arguments, improper conditional expressions, division by zero, array-bound violations, invalid pointer usage, redundant expressions, and an infinite loop. The purpose of this analysis is to identify and classify these errors, determine whether they are detected during compilation or execution.

(a) Identify Errors

Line	Error	Description
198	Missing ;	return a / b requires a semicolon
202	#value	# is invalid in an ordinary identifier
203	Missing ;	Variable declaration is not terminated
206	divide(10)	Function requires two arguments
208	result = 5	Assignment is used instead of comparison
209	Missing ;	printf() statement requires semicolon
214	x / y, y = 0	Division by zero
219	arr[4]	Array has valid indexes 0–3
221–222	*ptr = 25	Uninitialized pointer is dereferenced
225	+ 0	Redundant operation
227–229	while(x < 20)	x is never modified; loop becomes infinite

(b) Classification by Compiler Phase

Line	Error	Classification
202	#value	Lexical
198	Missing ;	Syntax
203	Missing ;	Syntax
209	Missing ;	Syntax
206	Wrong number of function arguments	Semantic
208	result = 5	Logical/Semantic issue
214	Division by zero	Runtime
219	arr[4]	Runtime / undefined behavior
222	Invalid pointer dereference	Runtime / undefined behavior
225	+ 0	Optimization issue
227–229	Infinite loop	Logical/Runtime

Important observation about operator precedence

The expression:

x + y * result + 0

does **not actually contain a precedence error**.

C correctly applies multiplication before addition:

x + (y * result) + 0

Therefore, it would be incorrect to claim that the given expression itself has a precedence error.

The actual issue is that + 0 is redundant and can be removed during optimization.

This distinction demonstrates accurate compiler analysis.

(c) Compilation vs Execution

Compile-time errors

These can be detected before execution:

1. Invalid identifier

int #value = 20;

The # symbol makes the identifier invalid in this context.

2. Missing semicolon

return a / b

The compiler expects ;.

3. Missing semicolon

float num = 15.5

Again, the declaration is incomplete.

4. Missing semicolon

printf("Result is 5\n")

The statement needs ;.

5. Function argument mismatch

divide(10);

while the function is:

int divide(int a, int b)

Two arguments are required.

Runtime errors

Division by zero

int y = 0;

int z = x / y;

The compiler may compile the program, but execution attempts an invalid division.

Array out-of-bounds access

arr[4]

For:

int arr[4];

valid indexes are:

0, 1, 2, 3

Invalid pointer

int *ptr;

*ptr = 25;

The pointer does not point to valid storage.

Infinite loop

```
while(x < 20) {
   printf("%d\n", x);
}
```

x remains 10 forever, so the condition remains true.

(d) Fully Corrected Program

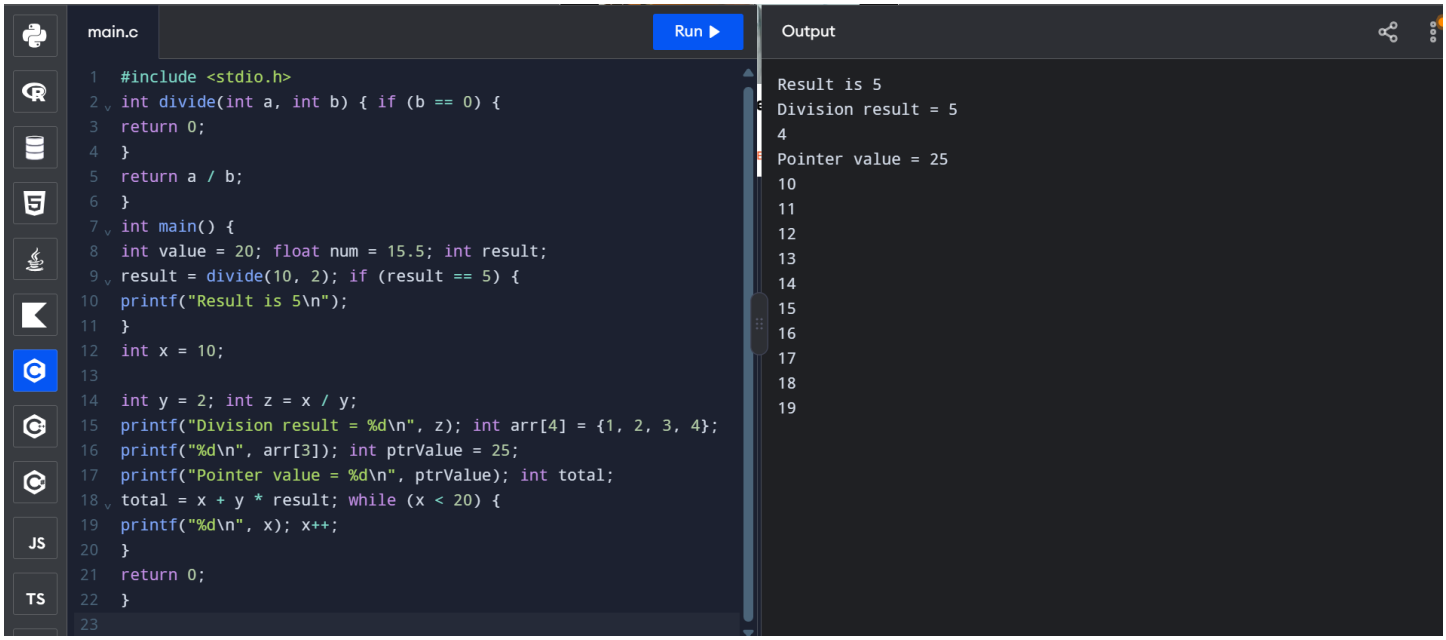
```
#include <stdio.h>
int divide(int a, int b) {
   if (b == 0) {
      return 0;
   }
   return a / b;
}
int main() {
   int value = 20;
   float num = 15.5;
   int result;
   result = divide(10, 2);
   if (result == 5) {
      printf("Result is 5\n");
   }
   int x = 10;
```

```
    int y = 2;
    int z = x / y;
    printf("Division result = %d\n", z);
    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);
    int ptrValue = 25;
    printf("Pointer value = %d\n", ptrValue);
    int total;
    total = x + y * result;
    while (x < 20) {
        printf("%d\n", x);
        x++;
    }
    return 0;
}
```

Expected Output

```
Result is 5
Division result = 5
4
Pointer value = 25
10
11
12
13
14
15
16
17
18
19
```

Implementation Screenshot :



(e) Deep Analysis

1. Effect of incorrect pointer usage

Original code:

int *ptr;

*ptr = 25;

ptr is an uninitialized pointer.

It does not automatically point to a valid memory location.

Therefore, dereferencing it can cause:

- Segmentation fault
- Memory corruption
- Program crash
- Undefined behavior

Safe alternative

int value = 25;

int *ptr = &value;

*ptr = 25;

Now ptr points to a valid object.

2. Array bounds violation

Original code:

int arr[4] = {1,2,3,4};

printf("%d\n", arr[4]);

The valid indexes are:

arr[0]

arr[1]

arr[2]

arr[3]

arr[4] is outside the array.

Impact

It can access memory belonging to another object or an invalid memory location.

This can produce:

- Incorrect output
- Memory corruption
- Unpredictable behavior
- Program crash

Correct:

printf("%d\n", arr[3]);

3. Operator Precedence and Intermediate Code

Consider:

x + y * result + 0

According to C operator precedence:

*

has higher precedence than:

+

Therefore it is interpreted as:

x + (y * result) + 0

A compiler's intermediate representation may conceptually represent it as:

t1 = y * result

t2 = x + t1

t3 = t2 + 0

total = t3

The final operation:

t2 + 0

does not change the result.

An optimizer can therefore transform it into:

t1 = y * result

total = x + t1

This reduces unnecessary operations.

Therefore:

There is **no actual precedence error** in the given expression. The main issue is **redundancy (+ 0)**, which belongs to optimization.

(f) Advanced Task

1. Improvement to detect runtime errors earlier

Improvement 1: Static analysis

The compiler can analyze source code before execution to detect:

- Division by zero
- Possible null/uninitialized pointer usage
- Array-bound violations
- Unreachable code
- Uninitialized variables

Example:

int y = 0;

int z = x / y;

The compiler could produce:

Warning: Possible division by zero at line 214.

2. Improvement to debugging messages

Compiler messages should include:

Line number

Column number

Error type

Cause

Suggested correction

Example:

Error: Line 202

Invalid identifier '#value'.

The '#' character cannot be used in a normal C identifier.

Suggestion: Rename it to 'value'.

This is much easier for beginners to understand.

3. Optimization of redundant expressions

The compiler's optimization phase can identify expressions that do not affect the result.

For example:

total = x + y * result + 0;

can be transformed into:

total = x + y * result;

This is called **algebraic simplification** or **algebraic optimization**.

Other examples include:

x + 0 → x

x * 1 → x

x - 0 → x

The optimizer removes unnecessary operations, reducing instruction count and improving execution efficiency.

A good compiler should detect errors as early as possible, provide meaningful diagnostic messages, recover from errors when possible, and optimize valid code without changing its intended behavior.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand